

Evaluating a Deterministic Validator Plus Single-Iteration Repair Pipeline for LLM-Generated Network Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (N=300 paired intent→configuration tasks; pre-registration id cand-2026-001-A-prereg-v1, pre-registration SHA-256 archived) to evaluate whether a minimal guarded pipeline—single-shot LLM generation (declared model: gpt-5-mini) followed by a frozen deterministic configuration validator and at most one automated repair iteration—reduces first-pass misconfiguration relative to plain single-shot generation. Execution completed 600 episodes and used 301 model calls. Observed outcomes: baseline = 299/300 valid (99.667%); guarded = 300/300 valid (100.0%); paired counts n11=299, n10=1, n01=0, n00=0. Exact McNemar two-sided $p = 1.00$; absolute paired difference = 0.0033333; paired bootstrap 95% CI = [0.00, 0.01] (10,000 resamples, seed 1729). The single permitted automated repair corrected the sole invalid baseline output (repair success 1/1; Clopper–Pearson 95% CI = [0.025, 1.0]). The preregistered planning assumption used for power calculations (planned baseline pass rate = 0.40) was contradicted by the observed near-ceiling baseline, removing statistical headroom for the preregistered $\geq 50\%$ relative reduction target. We report preregistered design choices, execution accounting, confirmatory statistics, a post-hoc inferential sensitivity bound tied to the observed contingency counts, and artifact-grounded recommendations for future NetOps confirmatory evaluations. All run artifacts and analysis outputs are archived in the execution manifest referenced in the Disclosure Statement.

I. INTRODUCTION

Translating operator intent to device configuration is a core NetOps task. Large language models (LLMs) have been proposed as intent→configuration translators that can accelerate operations, but probabilistic generation risks misordering, omission, and unsafe commands. Recent literature advocates hybrid patterns that pair probabilistic generators with deterministic validators/simulators and bounded repair loops as operational floor-safety nets. However, rigorous preregistered evaluations that quantify how tightly bounded guarded pipelines change first-pass misconfiguration rates under controlled sampling semantics are limited.

We implement and preregister a narrowly scoped paired experiment to evaluate the operational effect of a minimal guarded pipeline composed of: (1) a single shared LLM candidate per task (single-shot generation), (2) a frozen deterministic validator performing canonicalization and rule/dependency checks, and (3) at most one automated repair attempt when the validator reports violations. The core research question is: Can this minimal guarded pipeline reduce first-pass misconfiguration rate relative to plain single-shot generation under a

controlled paired design? Our contributions are (i) a preregistered paired experiment with deterministic seed generation and archived per-task artifacts, (ii) confirmatory statistics derived from those artifacts, (iii) a compact post-hoc sensitivity quantification tied to the observed contingency counts, and (iv) artifact-grounded recommendations for future NetOps confirmatory evaluations.

II. RELATED WORK

The recent surge of LLM applications to NetOps has produced three closely related strands of work relevant to guarded generate→verify→repair architectures. First, system demonstrations and intent-to-configuration translators show that LLMs can synthesize device configuration templates from natural language intent with promising accuracy in controlled settings, but these demonstrations frequently document ordering and omission errors that threaten operational safety [10.1002/nem.2313]. Those works typically evaluate end-to-end generation quality on curated testbeds and emphasize practical engineering concerns—prompt framing, template constraints, and device-family idiosyncrasies—rather than formalized confirmatory inference. Our experiment differs by preregistering paired inferential semantics and by measuring first-pass validity under a deterministic canonicalizer/validator and bounded repair budget.

Second, adjacent domains (medical QA, code repair, and software testing) provide concrete evidence that verifier-assisted pipelines, retrieval-augmented generation (RAG), and repair loops reduce factual errors and improve grounding relative to plain generation [10.36227/techrxiv.176784505.56675782/v1, 10.2139/ssrn.6608518]. Methodologically, these works combine (a) an LLM generator, (b) an external evidence or retrieval layer to ground outputs, and (c) a verifier or test harness that identifies discrepancies which then drive targeted regeneration or repairs. Crucially, reported gains vary with verifier fidelity and retrieval quality; effectiveness in one domain does not directly imply equivalent gains in NetOps where configuration correctness depends on strict, stateful device invariants and ordering constraints. Our guarded pipeline mirrors the generate→validate→repair motif but intentionally restricts intervention to a frozen deterministic validator and a single automated repair iteration to test a minimal operational floor-guard.

Third, the network-verification literature supplies algorithmic techniques for deterministic checking of syntactic and semantic properties of configurations. Header-space and related specification-based analyses enable static reachability and policy checks on canonicalized configuration representations and have been incorporated into tools for incremental verification and real-time policy checking [10.1145/3098822.3098834]. These deterministic techniques provide the computational primitives used by many recent NetOps validators: parse to an AST/canonical form, evaluate invariant and dependency rules, and flag violations deterministically. Our validator (deterministic_netops_validator_v5) follows that pattern—canonicalizing LLM outputs to normalized ASTs, checking required command sequences and workflow policies, and producing binary pass/fail scores plus coded violations—so the present study tests the practical effect of placing that class of verifier upstream of an automated, planner-guided single repair iteration.

Finally, survey and standards-style documents highlight system and governance concerns—hallucination, overconfidence, auditability, and human-oversight erosion—that motivate guarded architectures and reproducible evaluation protocols [10.6028/nist.ai.100-2e2023]. These works argue for verifiable logs, deterministic validators, and human-in-the-loop approval when systems affect critical assets. Our preregistered design and artifact archiving respond to these calls by (a) fixing pairing semantics and repair budgets before execution, (b) deterministically generating task seeds, and (c) archiving per-task normalized configurations, validator traces, and repair prompts/responses to support reproducible inference and forensic analysis.

Taken together, prior empirical and methodological work supports three practical expectations: verifier-assisted pipelines can reduce factual/configuration errors given a sufficiently capable verifier and repair strategy; deterministic static checks are appropriate first-line validators for many configuration properties; and rigorous evaluation of floor-guard pipelines requires careful alignment of sampling semantics, task difficulty, and preregistered power assumptions. The present study contributes by executing a tightly preregistered paired test that isolates the incremental effect of a frozen deterministic validator plus a single automated repair iteration on first-pass validity, documenting where those prior expectations are met and where preregistered planning assumptions (e.g., baseline pass rate = 0.40) interact with sampling semantics to limit inferential power.

III. METHODOLOGY

Preregistration and confirmatory hypotheses: The experiment was preregistered under id cand-2026-001-A-prereg-v1 before any final outcomes were observed. Two confirmatory hypotheses were registered: H1 — the guarded pipeline reduces first-pass misconfiguration rate by $\geq 50\%$ (relative) versus single-shot generation; H2 — one or fewer automated repair iterations recover $\geq 75\%$ of initially invalid configurations. The preregistration explicitly documented the numeric planning assumption used for power calculations:

planned baseline pass rate = 0.40. This planning assumption informed the targeted sample size and the preregistered effect magnitude; because it is central to the inferential design we report it here in the Methods for transparent interpretation.

Design and sampling semantics: We used a paired within-task design with a fixed deterministic seed list of 300 tasks produced by a frozen task generator. Each seed produced an intent and an associated canonical reference configuration; seeds were included deterministically unless a preregistered parser exclusion rule applied (such replacements are recorded in the archived manifest). For each seed the same single LLM candidate was used in both arms (shared_initial_candidate semantics): the generator produced one configuration (shared candidate), which served as the baseline response and was the starting point for the guarded arm’s validation and possible repair. Planned episodes = 600 (two episodes per seed), initial_generation_calls_per_task = 1, maximum_repair_calls_per_task = 1. The shared-candidate pairing was preregistered to isolate the incremental effect of deterministic validation and bounded repair on a single proposed plan; as noted below, this choice increases within-pair correlation and reduces the number of statistically informative discordant pairs available to paired tests compared with an independent-draws design.

Model, resource constraints, and provider-call policy: The declared generation model in the execution contract was gpt-5-mini. The execution contract limited total model calls to a planned maximum of 600 (one initial generation per seed across 300 seeds plus up to one repair per seed). Provider call failures are handled per the preregistered retry policy (single retry allowed for provider/infrastructure failures); the primary missingness treatment drops pairs only if both calls in a pair failed (‘complete_pair_primary’). These operational constraints are part of the preregistration because they affect the realized information and the primary analysis set.

Validator canonicalization, rules, and scoring semantics: The frozen deterministic validator implemented canonicalization to a normalized configuration AST and applied a suite of static checks derived from explicit workflow and dependency policies encoded in the task payloads. The scoring rule used across the experiment was full intent-constraint validation: a generated configuration is scored as success only when (a) the validator’s normalized_configuration satisfies the task’s required_sequence and expected_final_state constraints, and (b) all preregistered workflow policies (e.g., ‘must_be_down_for_fields’, ‘final_group_constraints’, and ‘minimum_active_in_groups’) are satisfied. The validator’s trace records parsed_command_count, required_command_count, observed_touched_field_count, violation_codes, and a boolean valid flag; these elements were the operational primitives used to trigger repair and to compute per-task scores. Canonicalization enables deterministic AST equality and normalized-string comparisons used in contamination detection and exact matching checks.

Repair adapter behavior and steering: The preregistered

automated repair adapter is invoked only when the validator reports violations on the shared candidate. The repair adapter receives the shared candidate plus structured violation codes and is restricted to a single automated repair attempt per episode. Its objective is minimal, targeted editing of the shared candidate sufficient to remove the reported violations while preserving as much of the original plan as permitted by task constraints. Repair prompts and repair responses were archived per episode; the repair outcome was revalidated by the same deterministic validator to produce the final guarded decision. Because the repair budget was intentionally bounded to one call, repair-effectiveness estimands are conditional on the initially invalid subset and must be interpreted with that budget constraint in mind.

Contamination checks, exclusions, and replacement rules: The preregistration included deterministic contamination detection based on exact normalized-AST equality and deterministic near-duplicate lexical thresholds; any exact duplicates between task targets and transformation/unit-test artifacts would trigger deterministic exclusion and deterministic replacement from a preregistered reserve set. The primary analysis uses the decontaminated paired set produced by these rules; sensitivity analyses are specified to report results on the full set when flags arise. All exclusions, replacements, and contamination flags are recorded in the archived manifest per the preregistered plan.

Primary estimand and statistical analysis plan: The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` (absolute difference in per-task success probability under guarded vs baseline). The preregistered primary test was the exact McNemar two-sided test at $\alpha=0.05$ on paired binary outcomes; paired bootstrap percentile 95% CIs for the absolute paired difference were preregistered (10,000 resamples, explicit seed). Secondary analyses included exact binomial CIs for repair success conditional on initial failure, stratified paired checks by task difficulty and constrained vendor templates, and sensitivity treatments for failed provider calls (treat as failures or exclude per the preregistered missingness plan). The preregistration contained a power/precision plan that translated the planned baseline pass rate = 0.40 and a target relative reduction ($\geq 50\%$ relative reduction in failure rate) into a required N under conservative discordant assumptions; that planning arithmetic is reported in the preregistration record and is discussed in the manuscript’s interpretation because the observed baseline diverged from that assumption.

Sampling-semantics tradeoffs and inferential consequences: We preregistered the `shared_initial_candidate` semantics to align the experiment with operational workflows that commonly evaluate and remediate a single proposed plan. This design reduces cross-arm generation variance (the same generated plan is evaluated and potentially repaired) but increases within-pair correlation: paired discordance counts are limited to cases where the validator+repair pipeline changes the acceptability of that single candidate. Consequently, the number

of informative discordant pairs available to McNemar inference is typically smaller than under independent-draws designs and must be accounted for when translating planned effect sizes into target sample sizes. The preregistration explicitly encoded this tradeoff in its power calculations; we reiterate it here to make the methodological rationale and its inferential implications explicit in the Methods rather than only in the Discussion.

Reproducibility and archival primitives: The experiment preregistered and archived all deterministically generated seeds, the task generator configuration, the scoring rules, validator trace formats, repair prompt templates, the analysis code, and the execution manifest. Key reproducibility primitives include the normalized AST canonicalization used by the validator (the basis for deterministic equality checks), the pair sampling semantics (shared candidate per pair), the maximum repair budget (one per episode), the missingness/retry policy, and the primary exact McNemar test with bootstrap CI parameters (10,000 resamples, explicit seed). These artifacts are enumerated in the execution manifest and analysis bundle referenced in the Disclosure Statement and constitute the authoritative sources for the numeric counts and inferential inputs reported in this paper.

IV. RESULTS

Execution accounting and artifact provenance (archived): The preregistered run completed all planned episodes: planned episodes = 600, completed episodes = 600, complete analyzed pairs = 300. Model calls consumed = 301 total (300 shared initial generation calls + 1 repair invocation). The archived deterministic reconciliation/provider audit records show no provider or infrastructure failures and no deterministic exclusions for contamination under the preregistered checks. These execution counts and stage summaries are recorded in the archived execution manifest and analysis bundle and are the direct source for all numeric values reported below.

Primary per-condition outcomes (archived counts): Baseline (single-shot) success count = 299/300 (99.6667%); Guarded (validator + ≤ 1 repair) success count = 300/300 (100.0%). The paired contingency table derived from the archived analysis artifact is: n11 (both success) = 299; n10 (guarded success, baseline fail) = 1; n01 (guarded fail, baseline success) = 0; n00 (both fail) = 0. Missingness summary in the archive reports zero failed or unscorable episodes in either arm under the preregistered missingness treatment.

Confirmatory inferential results (preregistered analyses executed on archived artifacts): The exact McNemar two-sided test on the observed contingency yields $p = 1.00$. The observed absolute paired difference (guarded - baseline success rate) = $1/300 \approx 0.003333$. Paired bootstrap percentile 95% CI for the absolute paired difference = [0.00, 0.01] (10,000 resamples, bootstrap seed = 1729), as computed in the archived analysis artifact. Repair outcomes: the single preregistered automated repair invocation succeeded on the sole initially invalid baseline output (repair success 1/1). The exact Clopper–Pearson

95% CI for a 1/1 repair success is [0.025, 1.0], reflecting the large uncertainty from a single observed repair case.

Why the McNemar $p = 1.00$ and what the discordant count implies: Exact McNemar inference is driven entirely by the discordant pair counts (n_{10} and n_{01}). With only one discordant pair observed ($n_{10} + n_{01} = 1$), the two-sided exact test cannot reject the null of no difference; the minimal nonzero discordant total yields an exact two-sided p of 1.00 under the test implementation used in the preregistered analysis. Equivalently, the discordant total places a hard upper bound on the resolvable absolute paired difference equal to discordants / 300 = $1/300 \approx 0.00333$. Put practically, the observed near-ceiling baseline left essentially no statistical headroom to detect the preregistered $\geq 50\%$ relative reduction (which, given the preregistration’s planning baseline = 0.40, mapped to an absolute change on the order of +0.30). Detecting an absolute +0.30 with $N=300$ would require on the order of tens of discordant pairs (the preregistration noted ~ 90 net discordant pairs aligned with the planned effect size under conservative assumptions); the archived discordant count of one is far below that requirement. These limits and numeric bounds are computed from the archived contingency artifact and bootstrap outputs.

Representative validator and diagnostic traces (artifact-grounded): For each episode the archived validator trace contains structured diagnostics used for scoring: `normalized_configuration`, `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes`, `validation_before/after` states, and a `repair_applied` flag. In the solitary baseline failure the validator trace documents the missing required command sequence and the canonicalized configuration before and after the automatic repair; the post-repair canonicalized configuration satisfied the full intent constraints per the validator. These per-task traces (validator decision fields and repair flags) are the primary provenance for the contingency counts and are archived alongside the analysis artifacts.

Sensitivity and interpretive bounds (post-hoc, artifact-grounded): Because the observed discordant total is one, the maximal absolute improvement observable in this dataset equals $1/300$. The paired bootstrap CI [0.00, 0.01] and the exact McNemar p together reflect that any larger effect size is incompatible with the observed discordant evidence. Operationally, this implies that confirmatory designs aiming to test large relative improvements must either (a) calibrate the task bank to produce a higher baseline failure prevalence or (b) adopt independent-draws sampling (increasing discordants at the cost of within-pair control) — choices that should be incorporated into power calculations. These precise numerical bounds are reproduced in the archived `paired_contingency_table.csv` and `paired_difference_figure.svg` contained in the analysis bundle.

Reproducibility and archived artifacts: All numerical counts, the bootstrap procedure (seed = 1729, resamples = 10,000), the exact-test implementation, and every per-task validator trace and repair prompt/response are included in the archived execution manifest and analysis bundle. The archived analysis

artifacts (condition summary, paired contingency table, bootstrap outputs, and deterministic reconciliation/provider audit) reproduce the figures and numbers above when loaded with the provided analysis executor. Readers who wish to reproduce the exact inferential outputs can consult the execution manifest and the bundled analysis artifacts referenced in the Disclosure Statement.

Concise summary of the results section: The guarded pipeline corrected the single observed invalid baseline output (artifact-grounded repair success = 1/1) but, because the baseline pass rate was far higher than the preregistration planning assumption, the run produced only one discordant pair and therefore lacked statistical information to verify the preregistered $\geq 50\%$ relative reduction. The run nevertheless demonstrates end-to-end execution, archival, and artifact-grounded diagnostics for a generate→validate→repair NetOps pipeline and clarifies concrete sample-size and sampling-semantic constraints necessary for informative confirmatory inference in future studies.

V. DISCUSSION

The guarded pipeline demonstrably remediated the single invalid baseline output observed in this run: the deterministic validator flagged the violation and the single permitted automated repair produced a canonicalized configuration that the validator accepted. This outcome empirically supports the practical utility of a frozen validator plus a tightly bounded repair step as a floor-safety mechanism that can convert some validator-detected failures into valid first-pass configurations.

However, the run also illustrates an important inferential and design caveat: confirmatory power for paired tests depends critically on the number of informative discordant pairs. Our preregistered `shared_initial_candidate` pairing was chosen to model operational workflows that examine and remediate a single generated plan. That design reduces within-pair heterogeneity and isolates the effect of validation+repair on a given candidate, but it also reduces the expected discordant pair count relative to independent draws per arm. In our execution the baseline pass rate was 299/300, producing only one discordant pair and thereby imposing a strict numerical ceiling on resolvable absolute improvement ($1/300 \approx 0.00333$). Consequently, the preregistered $\geq 50\%$ relative-reduction target—formulated under a planning assumption `baseline_pass_rate = 0.40`—became unattainable given the observed prevalences. The archived preregistration explicitly records the `baseline_pass_rate = 0.40` planning assumption and the analysis artifacts show the realized counts; together they explain why the confirmatory hypothesis could not be supported despite the guarded pipeline correcting the single failure it encountered.

Operationally useful lessons flow directly from these artifact-grounded facts. First, when baseline error prevalence is uncertain, researchers should design task banks or sampling procedures to produce an expected number of initial failures (or discordant pairs) aligned with the planned detectable effect. For the preregistered aim in this study, the analysis artifacts indicate that on the order of 90 discordant pairs would have

been needed to resolve an absolute change comparable with the preregistered target (≈ 0.30 absolute). In practice, investigators can (a) increase task difficulty, (b) include adversarial or stress-test augmentations, or (c) adopt stratified sampling by difficulty to concentrate statistical information where it is most informative. These are operational design options rather than claims about guaranteed outcomes; their selection should be preregistered and justified against expected discordant counts.

Second, choice of pairing semantics must be matched to the operational claim. If the goal is to measure whether validator+repair improves the probability that an operator will accept a generated plan when the generator is sampled afresh per condition, independent per-arm draws increase discordance and statistical information for McNemar-style inference. If, instead, the workflow examines and remediates a single proposed plan (the `shared_initial_candidate` semantics used here), then evaluation budgets should anticipate reduced discordant counts and compensate via deliberately higher-difficulty tasks or larger N . The preregistration tradeoff—higher within-pair control versus greater discordant information—should be made explicit in power computations; our archived power plan assumed `baseline_pass_rate` = 0.40 and thus did not account for the realized near-ceiling baseline.

Third, repair-effect estimands are inherently conditional on initial failures and require sufficient counts of those failures to estimate repair effectiveness precisely. In this run the observed repair success was 1/1, which is artifact-grounded but statistically imprecise (Clopper–Pearson 95% CI = [0.025, 1.0]). When repair effectiveness is a primary operational claim, studies should be powered to observe an adequate number of initial failures so that conditional repair success has narrow confidence intervals; appropriate sample sizing depends on the target precision and should be preregistered. The deterministic artifacts archived here (per-task validator traces and repair prompts) exemplify the level of per-case traceability required to support conditional estimands and subsequent diagnostics.

Finally, construct and external validity remain constrained in this study. Our validator performed deterministic static canonicalization and rule checks rather than full control-plane or packet-level emulation; therefore, runtime semantics and emergent control-plane interactions were not exercised. The task bank is synthetic and vendor-template coverage was constrained, and only one declared model (gpt-5-mini) and one validator implementation were used. These limitations narrow direct production generalization but do not diminish the methodological point: artifact-grounded preregistration combined with deterministic validator traces yields reproducible counts that clarify what was actually tested and which inferential questions remain open.

In sum, the empirical artifact shows that small guarded pipelines can correct individual validator-detected errors, but proving large relative reductions in misconfiguration rate under paired designs requires aligning task difficulty, pairing semantics, and sample size with the planned estimand. We therefore recommend that future confirmatory NetOps evaluations (a) preregister pairing semantics and the expected discordant-pair

count implied by the power calculation, (b) calibrate task difficulty or use adversarial augmentation to achieve a target initial-failure prevalence when necessary, and (c) archive per-task validator and repair traces to support transparent conditional estimands and post-hoc diagnostics. The archived execution and analysis artifacts accompanying this study provide a concrete example of how those recommendations can be operationalized.

VI. LIMITATIONS

- Synthetic task bank and constrained vendor-template scope limit external validity to broad production environments.
- Single LLM (gpt-5-mini) and a single validator implementation restrict generality across model families and validator designs.
- The preregistered power plan assumed a baseline pass rate = 0.40; the observed baseline (299/300) contradicts that assumption and removed headroom to detect the preregistered $\geq 50\%$ relative reduction.
- Sampling semantics (`shared_initial_candidate`) reused the same initial candidate across paired arms, increasing within-pair correlation and reducing discordant pairs available to McNemar inference.
- Validation used deterministic configuration/constraint checks rather than full dynamic emulation of runtime semantic effects; actual runtime consequences of configurations were not executed and remain unmeasured.
- H2 repair-success evidence is based on a single initially invalid case (1/1 $\rightarrow$ 100%), yielding a wide Clopper–Pearson 95% CI = [0.025, 1.0]; larger counts of initial failures are required to estimate repair effectiveness precisely.

VII. CONCLUSION

We executed a preregistered paired experiment ($N=300$ pairs) comparing single-shot LLM generation (gpt-5-mini) to a guarded pipeline consisting of a frozen deterministic validator and a single automated repair iteration. Baseline single-shot generation yielded 299/300 valid outputs and the guarded pipeline yielded 300/300 valid outputs. The single automated repair corrected the lone invalid baseline output, but the observed near-ceiling baseline contradicted the preregistered planning assumption (baseline pass rate = 0.40) and removed statistical headroom to detect the preregistered $\geq 50\%$ relative reduction. Future confirmatory NetOps evaluations should (a) calibrate task difficulty to produce sufficient initial failures or target discordant counts, (b) preregister pairing semantics and repair budgets explicitly, and (c) include emulation to measure runtime semantic effects when operational deployment claims are intended. All raw outputs, validator traces, repair traces, the execution manifest, and analysis artifacts are archived and enumerated in the Disclosure Statement to permit reproduction of the reported counts and intervals.

DISCLOSURE STATEMENT

This study was executed autonomously after the autonomy lock under the archived master prompt (provenance/master_prompt.txt; SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the master prompt prior to the autonomy lock; no manual human editing of manuscript technical content, figures, tables, or references occurred after the lock. Preregistration: cand-2026-001-A-prereg-v1 (the preregistration record was created before observing final outcomes; preregistration SHA-256 is recorded in the execution manifest). Declared model and tooling: gpt-5-mini (declared_model_version), adapter family hosted_netops_gvr_v1, frozen deterministic validator/repair adapter deterministic_netops_validator_v5. Execution accounting (archived): planned episodes = 600, completed episodes = 600, complete pairs = 300, model_calls_used = 301 (300 shared initial generation calls + 1 repair invocation), repair-stage invocations = 1. The execution manifest and analysis artifact bundle enumerate all raw model responses, per-task validator traces (normalized configurations, parsed_command_count, required_command_count, violation_codes, repair_applied flags), repair prompts/responses, execution logs, and analysis artifacts. The archived execution manifest and analysis bundle are the authoritative source for the numeric counts reported in this manuscript. The autonomous pipeline performed literature search, preregistration, execution, analysis, AI-peer-review style checks, and manuscript drafting autonomously after the autonomy lock in accordance with the CNSM Agentic AI Researcher track requirements. The preregistered planning assumption used in power calculations (planned baseline pass rate = 0.40) is explicitly reported here because it materially affects interpretation of the confirmatory result.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Oghenekeno Hilkiyah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1
- [3] Kunal Dhanda, "MedRAG: Retrieval-Augmented Generation for Medical QA-Comparing Base and RAG-Augmented LLM Performance on Evidence from Peer-Reviewed Clinical Research", 2026, 10.2139/ssrn.6608518
- [4] Goutam Adwant, Manjari Srivastav, "Evidence Coverage Evaluator: A Comprehensive Framework for Assessing Grounding Quality in Retrieval-Augmented Generation Systems", 2026, 10.36227/techrxiv.176784505.56675782/v1
- [5] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [6] Apostol Vassilev, Alina Oprea, Alie Jean Fordyce, Hyrum S. Anderson, "Adversarial machine learning :", 2024, 10.6028/nist.ai.100-2e2023